

AI CRM Brain

Deal Closure Prediction System

Engineering Roadmap v2.0 — Updated Architecture

Pattern MVC	Backend FastAPI + Python	Database PostgreSQL	Timeline 6 Sprints / 6 Weeks
----------------	-----------------------------	------------------------	---------------------------------

1. Executive Summary & Scope Changes

This document reflects the v2.0 architectural update to the AI CRM Brain — Deal Closure Prediction system. The following changes were ratified to maximise execution velocity, eliminate accidental complexity, and deliver a demo-ready MVP within the 6-sprint timeline.

Dropped Feature	Rationale
Real-time Webhook Triggers	Webhooks require public HTTPS endpoints, HMAC verification, and idempotency logic — over-engineered for MVP. Replaced by Active Batch Pull on a cron schedule.
LLM Fine-Tuning	Fine-tuning requires labelled preference datasets and GPU infrastructure. Few-Shot Prompting with Claude achieves equivalent output quality at zero additional cost.

✓ What This Means for Execution

The scope reductions above remove three of the highest-complexity engineering tasks from the critical path. The team can now focus 100% of Sprint 1 on a single, clean data source (Zoho), a single schema (3 PostgreSQL tables), and a single integration pattern (OAuth REST API).

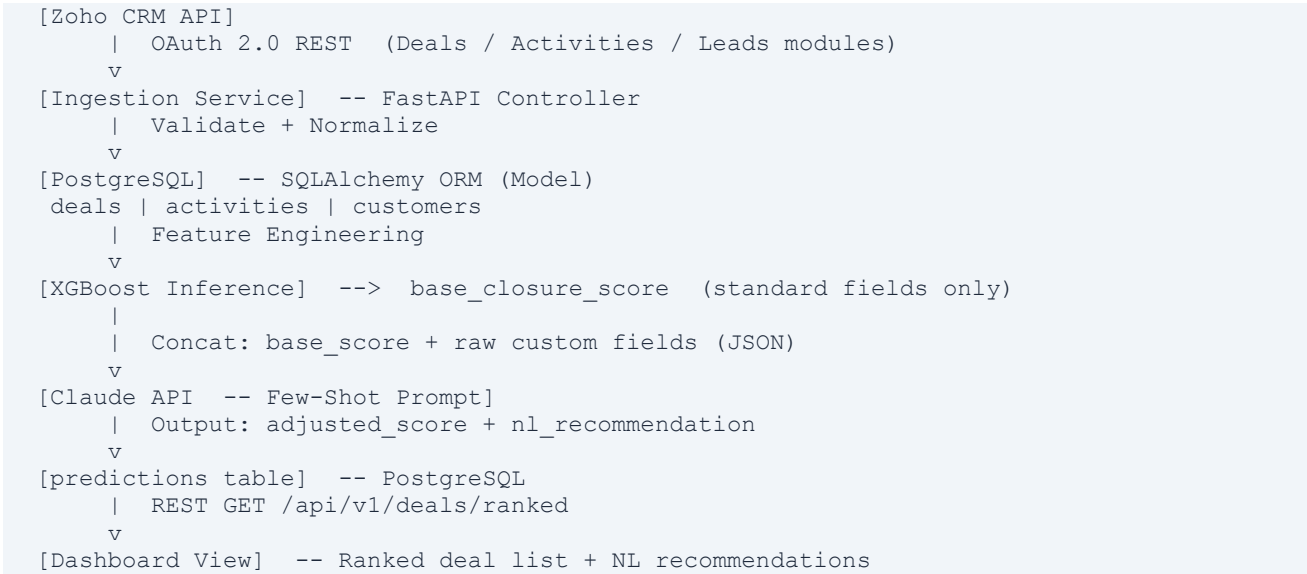
2. System Architecture — MVC Pattern

The system is structured strictly around the Model-View-Controller (MVC) pattern, applied to a FastAPI backend with a PostgreSQL persistence layer. This separation ensures each layer is independently testable and replaceable.

Layer	Component	Responsibility
Model	PostgreSQL + SQLAlchemy ORM	All data persistence: raw CRM records, engineered features, model predictions, recommendation logs.
Model	ML Layer (XGBoost / SHAP)	Deal closure probability inference. Runs as a Python service, reads from and writes back to the Model layer.

Layer	Component	Responsibility
Model	LLM Agent (Claude via API)	Receives base ML score + raw custom fields. Applies few-shot reasoning to produce final adjusted score and NL recommendation.
Controller	FastAPI Routers + Services	Orchestrates ingestion jobs, scoring pipeline, and recommendation generation. Exposes REST endpoints consumed by the View layer.
Controller	Active Batch Pull (APScheduler)	Periodic cron job: queries Zoho for Open/Active deals only, scores the full batch, writes results to DB, signals the View.
View	Dashboard (TBD — see §7)	Consumes FastAPI endpoints to display prioritised deal list, closure probabilities, and NL recommendations.

2.1 Component Interaction Diagram (Textual)



3. Data Infrastructure

3.1 CRM Data Source — Zoho CRM API

The MVP ingestion layer targets exactly three Zoho CRM modules. Strict module isolation guarantees structured JSON responses and eliminates schema ambiguity.

Zoho Module	Key Fields Ingested	Maps to PostgreSQL Table
Deals	Deal_Name, Amount, Stage, Closing_Date, Account_Name, Contact_Name, Deal_Owner, Probability, Lead_Source, custom_fields (stored as JSONB)	deals

Zoho Module	Key Fields Ingested	Maps to PostgreSQL Table
Activities	Activity_Type, Activity_Date, Initiated_By, Duration_Mins, Outcome_Flag, Related_Deal_ID	activities
Leads	Company_Size, Industry, Lead_Source, Decision_Maker_Role, Country, Annual_Revenue	customers

3.2 PostgreSQL Schema — Three Core Tables

```
-- Table 1: deals
CREATE TABLE deals (
  deal_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  zoho_deal_id     VARCHAR(64) UNIQUE NOT NULL,
  deal_name        TEXT,
  amount           NUMERIC(15,2),
  stage            VARCHAR(100),
  closing_date     DATE,
  account_name     TEXT,
  contact_name     TEXT,
  deal_owner       TEXT,
  probability       NUMERIC(5,2),
  lead_source      VARCHAR(100),
  is_closed_won    BOOLEAN,
  custom_fields    JSONB,           -- raw dump, NOT used in ML training
  ingested_at      TIMESTAMPTZ DEFAULT NOW()
);

-- Table 2: activities
CREATE TABLE activities (
  activity_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  deal_id          UUID REFERENCES deals(deal_id),
  activity_type     VARCHAR(64),
  activity_date    TIMESTAMPTZ,
  initiated_by     VARCHAR(20) CHECK (initiated_by IN ('rep','customer')),
  duration_mins    INT,
  outcome_flag     VARCHAR(64),
  ingested_at      TIMESTAMPTZ DEFAULT NOW()
);

-- Table 3: customers
CREATE TABLE customers (
  customer_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  zoho_lead_id     VARCHAR(64) UNIQUE,
  company_size     VARCHAR(50),
  industry         VARCHAR(100),
  decision_maker   VARCHAR(100),
  country          VARCHAR(100),
  annual_revenue   NUMERIC(15,2),
  ingested_at      TIMESTAMPTZ DEFAULT NOW()
);
```

pgAdmin 4 is the designated GUI for schema inspection, query execution, and manual data validation during development. All schema migrations are managed via Alembic.

3.3 OAuth 2.0 Integration — Zoho CRM

- **Step 1:** Register the application in the Zoho API Console to obtain Client ID and Client Secret.
- **Step 2:** Implement the Authorization Code Grant flow. Store the refresh token securely in environment variables (never in source code).
- **Step 3:** FastAPI dependency injection pattern: a ZohoCRMClient class handles token refresh automatically before each API call.
- **Step 4:** Rate limit awareness: Zoho enforces a per-org API call limit. The ingestion service must implement exponential backoff and respect the X-RATELIMIT-REMAINING response header.

3.4 Data Validation Layer

Every record passes through a three-stage validation pipeline before being written to PostgreSQL. Records failing any stage are routed to a flagged_records table and excluded from model input.

Stage	Check Type	Validation Logic
V-1	Null / Missing Fields	Flag records where Amount IS NULL, Stage IS NULL, or Closing_Date IS NULL. These are required for ML inference.
V-2	Type Consistency	Assert Amount is NUMERIC, Closing_Date parses to ISO 8601 (YYYY-MM-DD), Probability is 0.0-100.0 range.
V-3	Date Sanity	Closing_Date must not be in the past by more than 2 years (historical anomaly) or more than 5 years in the future (data entry error). Flag outliers.
Report	Data Quality Report	After each ingestion run, generate a JSON + text summary: total records, pass rate, failure breakdown by check type, top offending fields.

4. Machine Learning & AI Strategy

4.1 Core ML Model — Deal Closure Prediction

The ML model trains exclusively on fixed, system-defined standard fields. This constraint is non-negotiable for the MVP and is the primary defence against matrix mismatch errors during inference.

Training Feature Set (Standard Fields Only)	Explicit Exclusions
deal_cycle_days (Close_Date - Created_Date)	All JSONB custom_fields column content
amount_log (log-transform of Amount)	User-defined Zoho custom fields of any type
stage_encoded (ordinal: 1=Discovery to 5=Contract)	Free-text fields (descriptions, notes)
days_in_current_stage (compared to historical avg)	Any field not present in the standard Zoho Deals schema
probability (Zoho native estimate, 0-100)	
lead_source_encoded (one-hot)	

Training Feature Set (Standard Fields Only)	Explicit Exclusions
total_activities (count from activities table)	
customer_initiation_ratio (% customer-initiated)	
avg_response_lag_days	

Missing Value Strategy — Data Imputation

- **Numeric:** Numeric fields (amount, probability): impute with the median value computed on the training set.
- **Categorical:** Categorical fields (lead_source, industry): impute with the string literal "Unknown" before one-hot encoding.
- **Date-derived:** Date-derived fields (days_in_current_stage): impute with the mean stage duration from historical closed deals.

Imputation is fitted on the training set only and applied at inference time via a serialised scikit-learn Pipeline object. This prevents data leakage.

Target Variable & Model Selection

- Target: is_closed_won (binary: 1 = Closed-Won, 0 = Closed-Lost).
- Algorithm: XGBoost Classifier (primary) with Logistic Regression as an interpretable baseline for sanity-checking.
- Evaluation: AUC-ROC (primary metric), Precision-Recall curve (given likely class imbalance), and F1 score.
- Train/test split: TIME-BASED only. Train on deals closed before a cutoff date; test on deals closed after. Never random split on time-series sales data.
- Class imbalance handling: XGBoost scale_pos_weight parameter, tuned to the ratio of negative to positive samples.

4.2 LLM Recommendation Engine — Few-Shot Prompting Strategy

The system does NOT fine-tune an LLM. Instead, it leverages Claude 3.5 Sonnet via the Anthropic API, providing context and examples directly in the prompt at inference time. This approach requires zero additional training infrastructure and delivers equivalent reasoning quality for this task.

Prompt Structure

System Prompt:

You are a senior CRM sales analyst. Given a base deal closure probability score and supplementary deal fields, output: (1) an adjusted final score 0-100 and (2) a 2-3 sentence actionable recommendation for the sales representative.

Few-Shot Examples (injected in user turn):

3-5 real closed deal examples with known outcomes, structured as JSON. These calibrate the LLM's reasoning to your specific sales domain.

Live Payload (per deal):

```
{ "base_score": 0.72, "stage": "Negotiation",
  "custom_fields": { "Budget_Confirmed": "Yes",
    "Competitor_Present": "Oracle" }, "days_to_close": 12 }
```

The LLM acts as a logic reasoner: it receives structured data it did not train on (custom fields), applies the context provided by few-shot examples, and returns a calibrated final score plus a human-readable recommendation. This is the final output surface seen by the sales manager on the dashboard.

4.3 Scoring Pipeline — End-to-End Flow

```

1. FastAPI triggers Active Batch Pull (cron schedule)
2. Zoho API queried: GET /crm/v3/Deals?status=open&fields=...
3. Validation Layer: V-1 → V-2 → V-3 checks applied
4. Standard features extracted → imputation pipeline applied
5. XGBoost.predict_proba() → base_score per deal
6. For each deal:
    payload = { base_score, stage, days_to_close,
                custom_fields (raw JSONB from deals table) }
    claude_response = anthropic.messages.create(
        model="claude-sonnet-4-5",
        system=SYSTEM_PROMPT,
        messages=[few_shot_examples + payload]
    )
7. Parse claude_response → adjusted_score, nl_recommendation
8. Write to predictions table:
    (deal_id, base_score, adjusted_score, recommendation, scored_at)
9. FastAPI GET /api/v1/deals/ranked returns sorted predictions
10. Dashboard View renders ranked deal list

```

5. Workflow & Trigger Architecture — Active Batch Pull

⚠ Architecture Decision: Dropping Real-Time Webhooks

Webhooks require a publicly reachable HTTPS endpoint, HMAC signature validation, at-least-once delivery guarantees, and idempotency logic — a significant infrastructure overhead for an MVP. The Active Batch Pull pattern delivers equivalent business value (fresh predictions every N minutes) with a fraction of the complexity.

The Active Batch Pull is implemented as an APScheduler background task registered at FastAPI startup. It executes on a configurable interval (default: every 30 minutes during business hours).

```

from apscheduler.schedulers.asyncio import AsyncIOScheduler

scheduler = AsyncIOScheduler()

@scheduler.scheduled_job("interval", minutes=30)
async def active_batch_pull():
    """
    1. Query Zoho: open deals modified in last 24h
    2. Run validation pipeline
    3. Score batch via XGBoost + Claude
    4. Upsert predictions table
    5. Log run metadata (duration, records processed, error count)
    """
    deals = await zoho_client.get_open_deals(modified_since=last_run)
    validated = validation_pipeline.run(deals)
    predictions = await scoring_service.score_batch(validated)
    await db.upsert_predictions(predictions)

scheduler.start()

```

The batch pull queries only Open/Active deals (not historical closed deals, which are already in the training set). The entire batch is scored simultaneously in a single XGBoost inference call (vectorised), then individual LLM calls are issued per deal. LLM calls can be parallelised using `asyncio.gather()` subject to API rate limits.

6. Tech Stack — Full Specification

Phase	Category	Tool / Library	Version Target
Data	CRM Source	Zoho CRM REST API v3	OAuth 2.0
Data	HTTP Client	httpx (async)	^0.27
Data	Database	PostgreSQL	16.x
Data	ORM	SQLAlchemy (async)	^2.0
Data	Migrations	Alembic	^1.13
Data	DB Admin GUI	pgAdmin 4	Latest
Feature Eng.	Data Processing	pandas, numpy	^2.x, ^1.x
Feature Eng.	Preprocessing	scikit-learn Pipelines	^1.5
Feature Eng.	Imbalance	imbalanced-learn (SMOTE)	^0.12
ML	Classifier	XGBoost	^2.0
ML	Explainability	SHAP	^0.46
ML	Experiment Tracking	MLflow	^2.13
LLM	AI Provider	Anthropic SDK (Claude)	^0.30
LLM	Orchestration	LangChain (prompt templates)	^0.2
Backend	API Framework	FastAPI	^0.111
Backend	ASGI Server	Uvicorn	^0.30
Backend	Data Validation	Pydantic v2	^2.7
Backend	Task Scheduler	APScheduler	^3.10
Backend	Testing	pytest + httpx	^8.x
DevOps	Containerisation	Docker + Docker Compose	Latest
DevOps	Config Management	python-dotenv + .env	^1.0
View	Dashboard	TBD — See Section 7	—

7. Sprint Timeline — 6-Week Execution Plan

Sprint 1 · Discovery & Data Infrastructure

Week 1 · Foundations: Zoho API → PostgreSQL Pipeline

Goals

- Establish the complete MVC directory structure and FastAPI project scaffold.
- Deploy PostgreSQL locally with Docker and configure pgAdmin 4 for schema management.
- Implement all three SQLAlchemy ORM models (deals, activities, customers) and run initial Alembic migration.
- Build the ZohoCRMClient class with OAuth 2.0 Authorization Code flow, automatic token refresh, and rate limit backoff.
- Implement the three-stage Data Validation Layer (V-1 null checks, V-2 type assertions, V-3 date sanity) with routing to flagged_records table.
- Write and execute the Data Quality Report script: outputs a JSON summary of ingestion run health.

✓ Sprint 1 Deliverable

A fully operational data pipeline: OAuth-authenticated Zoho API connection → Validation Layer → PostgreSQL (three normalised tables). The pipeline must ingest at least one full pull of open deals, pass validation, and confirm clean records visible in pgAdmin 4. A Data Quality Report JSON file is generated post-run.

Sprint 2 · Feature Engineering & EDA

Week 2 · Data Transformation & ML-Ready Feature Sets

Goals

- Implement all standard field feature engineering transformations as a scikit-learn Pipeline (deal_cycle_days, amount_log, stage_encoded, engagement ratios, etc.).
- Fit and serialise the imputation pipeline (median for numeric, "Unknown" for categorical) on historical training data.
- Conduct EDA: distribution analysis, correlation heatmaps, class balance audit for the is_closed_won target.
- Implement time-based train/test split logic. Define the cutoff date and validate that no leakage exists.
- Apply SMOTE or XGBoost scale_pos_weight if class imbalance exceeds a 3:1 ratio.
- Document all engineered features in a Feature Dictionary markdown file.

DELIVERABLE: Cleaned, imputed, time-split feature matrix (**X_train**, **X_test**, **y_train**, **y_test**) serialised to disk, plus EDA notebook with key observations.

Sprint 3 · Core ML Model Development

Week 3 · XGBoost Training, Evaluation & SHAP

Goals

- Train the XGBoost Deal Closure Prediction model on the time-split training set.
- Evaluate against test set: AUC-ROC, Precision-Recall curve, F1. Document results in MLflow.
- Train Logistic Regression baseline for comparison and interpretability sanity check.
- Run SHAP analysis on the XGBoost model. Produce a SHAP summary plot and identify the top 5 closure-driving features.
- Register the best-performing model in the MLflow Model Registry.
- Expose model inference via a FastAPI endpoint: POST /api/v1/predict (single deal payload).

DELIVERABLE: Trained, evaluated, and registered XGBoost model. FastAPI prediction endpoint passing pytest integration tests. SHAP feature importance report.

Sprint 4 · LLM Layer & Active Batch Pull

Week 4 · Few-Shot Prompting + Scheduler

Goals

- Design and test the few-shot prompt template using 3-5 real closed deal examples. Validate LLM output structure (JSON: adjusted_score, nl_recommendation).
- Build the ScoringService class that orchestrates the full pipeline: XGBoost base score → payload construction → Claude API call → response parsing.
- Implement the Active Batch Pull APScheduler job. Configure interval, scope (open deals only), and error handling / retry logic.
- Create the predictions table in PostgreSQL. Implement upsert logic to avoid duplicate prediction records per deal per run.
- Expose the ranked deal endpoint: GET /api/v1/deals/ranked (returns predictions sorted by adjusted_score DESC).

DELIVERABLE: End-to-end scoring pipeline operational. APScheduler running on 30-minute intervals. Ranked deal list accessible via REST API. All pipeline stages covered by pytest unit tests.

Sprint 5 · Dashboard & Integration Testing

Week 5 · View Layer + Full System QA

? Open Decision: Dashboard Implementation — Your Input Required

The View layer implementation method has not been finalised. Two viable options are presented below. This decision impacts Sprint 5 effort significantly and must be confirmed before development begins. See Section 8 for full analysis.

Goals (Dashboard-Agnostic)

- Conduct end-to-end integration testing: Zoho pull → validation → scoring → API → View.

- Performance test: full batch scoring latency must be under 5 seconds for up to 500 active deals (XGBoost inference is vectorised; LLM calls are the bottleneck and should be parallelised with `asyncio.gather`).
- Implement the dashboard View consuming `GET /api/v1/deals/ranked`. Minimum viable panels: ranked deal list, closure probability badge, engagement temperature, NL recommendation text.
- Conduct user simulation with a sales manager persona. Validate recommendation clarity and ranking logic.

DELIVERABLE: Fully integrated system demo-ready. A sales manager can view the ranked deal list and read actionable NL recommendations. All critical paths covered by integration tests.

Sprint 6 · Demo, Documentation & Delivery

Week 6 · Polish, Pitch & Handoff

Goals

- Record the 5-10 minute product demo video.
- Write technical documentation: architecture overview, API endpoint reference (OpenAPI/Swagger auto-generated by FastAPI), model cards for the XGBoost model.
- Write the user guide: connecting Zoho, reading the dashboard, interpreting recommendations.
- Prepare the Business Case: model revenue uplift from improved close rate (e.g., 22% → 28% on a \$2M pipeline = \$120K additional revenue per quarter).
- Finalise the investor/client pitch deck: Problem → Solution → Live Demo → Market Size → Competitive Advantage.

DELIVERABLE: Complete, polished, demo-ready system. Full documentation suite. Final pitch deck. All code committed to version control with a clean README.

8. Open Decision: Dashboard Implementation (Sprint 5)

The dashboard is the system's primary user-facing surface — the difference between a working API and a working product. Two implementation options are presented below. Each has materially different engineering effort and presentation quality implications.

	Dimension	Option A: Streamlit	Option B: React / Vue SPA
	Dev effort	Low (1-2 days for MVP panels)	High (3-5 days minimum for MVP)
	Visual quality	Functional but identifiably "data-science tool"	Production-grade, fully customisable
	FastAPI integration	Direct Python calls or requests to API	REST API calls via <code>fetch</code> / <code>axios</code>
	Demo impression	Suitable for technical judges	Significantly stronger for investor/client pitch

	Dimension	Option A: Streamlit	Option B: React / Vue SPA
	Real-time updates	Limited (requires st.rerun or polling hack)	Native (polling, SSE, or WebSocket)
	Recommended for	Solo developer, time-constrained MVP	Team with frontend capacity, pitch-critical demo

🔗 Clarifying Question — Action Required Before Sprint 5

Which dashboard implementation do you want to proceed with? Option A (Streamlit): Fastest path to a working visual. Best choice if Sprint 5 is time-constrained or if the primary audience is technical evaluators. Option B (React/Vue SPA): Highest demo impact. Best choice if the final presentation targets investors, clients, or a competition panel evaluating production readiness. Your answer determines the Sprint 5 task breakdown and the frontend tech stack entries in Section 6. Please confirm before Week 4 ends.

9. Architectural & Professional Recommendations

9.1 Environment Configuration

- All secrets (Zoho Client ID/Secret, Refresh Token, Anthropic API Key, PostgreSQL DSN) are loaded exclusively from environment variables via python-dotenv.
- Maintain three .env files: .env.development, .env.test, .env.production. Never commit any .env file to version control. Add *.env to .gitignore immediately.
- Use Docker Compose to run PostgreSQL and the FastAPI app as co-located services from day one. This eliminates "works on my machine" issues across the team.

9.2 The Custom Fields Architecture Decision

Storing custom_fields as JSONB in PostgreSQL is a deliberate and correct architectural choice. It achieves three objectives simultaneously:

- It guarantees the standard ML feature matrix has a fixed, predictable shape (no matrix mismatch errors).
- It preserves the raw custom field data for the LLM agent without any schema changes.
- It creates a clear upgrade path: in v2.0, high-cardinality custom fields can be analysed for predictive power and promoted to standard ML features if warranted by the data.

9.3 Few-Shot Prompt Calibration

The quality of the LLM's adjusted score and recommendation depends entirely on the quality of the few-shot examples injected into the prompt. These examples must be curated from real, verified closed deals in the client's Zoho instance. Key practices:

- Include both Closed-Won and Closed-Lost examples to prevent the model from being systematically optimistic.
- Vary the examples across industries, deal sizes, and stages.
- Specify the exact output JSON schema in the system prompt and validate the LLM response against a Pydantic model before writing to the database.

- If the LLM returns an invalid response, implement a retry with a corrective prompt (max 2 retries) before falling back to the base ML score only.

9.4 MLOps & Model Lifecycle

- Use MLflow to log every training run: hyperparameters, AUC-ROC, F1, dataset hash, and model artifact.
- Implement a monthly automated retraining job (APScheduler) that pulls the last 12 months of closed deals, retrains XGBoost, compares AUC-ROC against the current production model, and promotes the new model only if it outperforms.
- The model version number is stored in the predictions table alongside each scored record, enabling regression analysis if a new model version underperforms.

9.5 Scalability Path

The MVP architecture is intentionally simple. The following upgrade path is defined for post-MVP scale:

- LLM parallelism: Replace sequential `asyncio.gather()` with a proper async task queue (Celery + Redis) when deal volume exceeds 1,000 per batch.
- Database: Add a read replica for the dashboard query path when write traffic from ingestion jobs impacts dashboard latency.
- Multi-tenancy: The deals table can be extended with a `tenant_id` UUID column and Row-Level Security (RLS) policies in PostgreSQL to isolate client data at the database layer.
- Model serving: Migrate XGBoost inference from in-process Python to a dedicated FastAPI microservice when prediction latency needs to be independently scaled.